instituto
superior de
engenharia
de lisboa

Sistema de interface de dados para a CIÊNCIAVITAE

Beatriz Filipe

Orientadores: Diego Passos
Fernanda Passos

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2025

Instituto Superior de Engenharia de Lisboa

Sistema de interface de dados para a CIÊNCIAVITAE

49751 Beatriz Maria Rocha Filipe

Orientadores: Diego Passos

Fernanda Passos

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2025

Resumo

O principal objetivo deste projeto é facilitar a gestão e a interoperabilidade de currículos científicos, tendo como domínio central a plataforma *CIÊNCIAVITAE*, através de uma aplicação web que permite importar, editar e exportar currículos científicos de forma simples e estruturada.

A aplicação oferece aos utilizadores a possibilidade de importar os seus currículos científicos a partir de plataformas externas, visualizar e editar os dados extraídos, e exportá-los para a CIÊNCIAVITAE através da sua API (Interface de Programação de Aplicações) oficial. O sistema garante ainda o armazenamento organizado dos dados numa base de dados relacional, permitindo uma gestão eficiente e fiável da informação.

Para demonstrar a viabilidade da solução, foi utilizado como estudo de caso o *Lattes*, uma plataforma utilizada no Brasil para gestão de currículos de investigadores. Esta plataforma fornece os dados dos currículos em formato XML.

Para além disso, o sistema foi concebido com uma arquitetura extensível e modular, com o objetivo de permitir, no futuro, a integração com outras plataformas de gestão de currículos científicos. Esta aplicação visa apoiar investigadores, instituições de ensino e outras entidades na gestão eficiente dos seus currículos científicos, ajudando a evitar a duplicação de informações, a poupar tempo em processos manuais e a aumentar a fiabilidade dos dados.

Com este projeto, foram aprofundados conhecimentos no desenvolvimento de aplicações web, na transformação e normalização de dados, bem como na integração de sistemas com APIs externas, consolidando competências técnicas com aplicação prática em diversos contextos tecnológicos.

Palavras-chave: gestão de currículos científicos; interoperabilidade; aplicação web; arquitetura extensível; Plataforma *CIÊNCIAVITAE*; Plataforma *Lattes*

Índice

1 Introdução **1**

1.1 Motivação 1

1.2 Requisitos Funcionais 2

1.2.1 Requisitos Obrigatórios 2

1.2.2 Requisitos Opcionais 2

1.3 Requisitos Não Funcionais 2

1.4 Organização do documento 3

2 Enquadramento **4**

2.1 Trabalhos relacionados 4

2.2 Sistemas semelhantes 5

3 Arquitetura **6**

3.1 Diagrama de Blocos 6

3.2 Casos de Uso 7

3.2.1 Casos de Uso de Autenticação 7

3.2.2 Casos de Uso de Gestão de Currículo 8

3.3 Modelo de Dados 9

4 Tecnologias **14**

4.1 *Backend* 14

4.1.1 Kotlin 14

4.1.2 Spring 15

4.1.3 Jackson XML 15

4.1.4 PostgreSQL 15

4.2 Frontend Web 16

4.2.1 TypeScript 16

4.2.2 React 16

4.3 Deployment 17

4.3.1 Docker 17

4.3.2 Docker Compose 17

4.4 Suporte ao Desenvolvimento . . . 17
4.4.1 Git e GitHub . . . 17
4.4.2 Postman . . . 17
4.4.3 Dia e diagrams.net . . . 18

5 Desenvolvimento **19**
5.1 Metodologia de Desenvolvimento . . . 19
5.2 Estrutura do Projeto . . . 19
5.2.1 Estrutura de Diretórios . . . 20
5.2.2 Endpoints REST . . . 21
5.3 Autenticação e Gestão de Utilizadores . . . 22
5.3.1 Registo de Utilizadores . . . 22
5.3.2 Login de Utilizadores . . . 22
5.3.3 Logout de Utilizadores . . . 23
5.3.4 Filtro de Autenticação . . . 23
5.3.5 Configuração de Segurança . . . 24
5.4 Processamento e Integração de Currículos (CVs) . . . 24
5.4.1 Importação de CVs . . . 24
5.4.2 Implementação do *LattesCvMapper* . . . 26
5.4.3 Exportação para a plataforma *CIÊNCIAVITAE* . . . 28
5.5 *Frontend Web* . . . 30

6 Testes **32**
6.1 Ferramentas Utilizadas . . . 32
6.2 Conclusão . . . 33

7 Conclusão **34**

Referências **38**

A Base de Dados **39**

Lista de Figuras

3.1 Diagrama de blocos de alto nível da aplicação 6
3.2 Casos de Uso de Autenticação 7
3.3 Casos de Uso de Gestão de Currículo 8
3.4 Visão simplificada do diagrama EA com as principais entidades. 9

5.1 Estrutura de diretórios do projeto 20
5.2 Geração do *token* de autenticação 23
5.3 Fluxo do filtro de autenticação por *token* 24
5.4 Diagrama de classes da arquitetura de importação de CVs 25
5.5 Exemplo do processo de mapeamento – Secção de Idiomas 27
5.6 Diagrama da exportação de competências linguísticas para a CIÊNCIAVITAE. 29
5.7 Fluxo de navegação da interface do utilizador 31

A.1 Diagrama Entidade-Associação do projeto. 40

Capítulo 1

Introdução

Neste capítulo, é apresentada a motivação que esteve na origem deste projeto, os principais requisitos funcionais e não funcionais, bem como a forma como este relatório está organizado.

1.1 Motivação

Atualmente, a investigação científica e a atividade académica dependem, cada vez mais, de sistemas estruturados e interoperáveis para a gestão e divulgação de currículos científicos. Plataformas como a *CIÊNCIAVITAE* [1] desempenham um papel fundamental neste contexto, ao permitir que os investigadores registem e mantenham os seus percursos académicos, profissionais e científicos de forma organizada e acessível.

Contudo, apesar da sua utilidade, a *CIÊNCIAVITAE* apresenta limitações no que diz respeito à interoperabilidade com outras plataformas. Embora ofereça suporte à importação de dados a partir de algumas fontes externas — como será discutido na Secção 2.2 — continuam a existir limitações na integração com outras plataformas amplamente utilizadas. Esta limitação obriga frequentemente à introdução redundante de dados ou à realização de procedimentos manuais demorados para a migração ou sincronização de informação entre sistemas. Esta realidade origina ineficiências, inconsistências nos dados e um esforço adicional por parte dos investigadores e das instituições.

Neste contexto, este trabalho apresenta o desenvolvimento de um sistema de interface de dados *web*, com o objetivo de facilitar a integração e a interoperabilidade entre a *CIÊNCIAVITAE* e outras plataformas de currículos científicos, como o *Lattes* [2], uma plataforma consolidada no Brasil utilizada para a gestão de currículos científicos. Através desta solução, pretende-se permitir aos utilizadores importar, editar e exportar os seus currículos científicos de forma estruturada e intuitiva, promovendo a automação, reduzindo redundâncias e melhorando a qualidade da informação.

A aplicação desenvolvida integra um serviço *backend* responsável pelo tratamento e armazenamento dos dados, e uma interface *frontend*, que proporciona ao utilizador uma experiência clara e funcional para visualizar e modificar os seus dados com facilidade e eficiência.

O sistema foi concebido com foco na modularidade e extensibilidade, permitindo futuras integrações com outras plataformas e normas utilizadas na gestão de informação científica.

1.2 Requisitos Funcionais

Os requisitos funcionais deste projeto definem o conjunto de funcionalidades que o sistema deve disponibilizar para alcançar os seus objetivos principais. Durante a fase de conceção, foram definidos um conjunto de requisitos obrigatórios, considerados essenciais para o funcionamento base do sistema, e um conjunto de requisitos opcionais, cuja implementação seria considerada um valor acrescentado, a realizar caso o tempo de desenvolvimento o permitisse.

1.2.1 Requisitos Obrigatórios

- Desenvolver um sistema de gestão de utilizadores com autenticação.
- Permitir a importação de currículos científicos a partir da plataforma *Lattes*.
- Converter os dados importados para um modelo interno de CV independente da origem, que sirva como formato intermédio para visualização, edição e exportação (ver Secção 3.3).
- Disponibilizar uma interface de edição que permita ao utilizador modificar os dados importados antes de proceder à exportação.
- Exportar os currículos para a plataforma *CIÊNCIAVITAE*, utilizando a sua API oficial [3].
- Armazenar os dados do currículo científico de forma estruturada numa base de dados relacional.
- Garantir uma arquitetura extensível que permita, no futuro, integrar novas plataformas de gestão de currículos científicos.

1.2.2 Requisitos Opcionais

- Permitir a importação de currículos científicos da plataforma *CIÊNCIAVITAE* para o sistema, utilizando a sua API oficial.

1.3 Requisitos Não Funcionais

Para além das funcionalidades descritas anteriormente, também foi definido um conjunto de requisitos não funcionais, que estabelecem critérios de qualidade como a facilidade de uso, o bom desempenho, a facilidade de manutenção e a confiança no funcionamento do sistema.

- Desenvolver uma interface de utilizador simples, responsiva e intuitiva.

- Assegurar uma boa organização e separação dos componentes do sistema (*frontend*, *backend* e base de dados), de acordo com boas práticas de engenharia de *software*.
- Incluir testes automatizados para garantir a robustez e fiabilidade das funcionalidades implementadas.

1.4 Organização do documento

O relatório está estruturado em 7 capítulos e encontra-se organizado da seguinte forma: O Capítulo 2 (Enquadramento) apresenta o enquadramento do projeto, incluindo os trabalhos relacionados. O Capítulo 3 (Arquitetura) descreve a arquitetura da aplicação. O capítulo 4 (Tecnologias) aborda as tecnologias utilizadas no projeto. Segue-se o Capítulo 5 (Desenvolvimento), que detalha os principais aspetos da implementação. No Capítulo 6 (Testes), é descrita a estratégia de testes adotada. Por fim, o Capítulo 7 (Conclusão) apresenta as principais conclusões do projeto e propõe direções para trabalho futuro.

Capítulo 2

Enquadramento

Este capítulo está organizado em duas secções onde se descrevem os trabalhos relacionados e alguns sistemas semelhantes ao sistema desenvolvido.

2.1 Trabalhos relacionados

A gestão de informação científica tem vindo a enfrentar diversos desafios, entre os quais se destacam a necessidade de interoperabilidade entre plataformas e a normalização de dados. Iniciativas como o ORCID (Open Researcher and Contributor ID) [4], contribuem para a padronização da identificação de autores e facilitam a sincronização de dados entre diferentes sistemas de informação científica, tais como repositórios institucionais e sistemas de currículos científicos.

Em Portugal, o PTCRIS (Plataforma de Gestão Integrada da Informação Científica e Tecnológica) [5] tem promovido a integração dos sistemas de informação de suporte à atividade científica num ecossistema único, coerente e integrado, através da adoção de normas comuns, desenvolvendo serviços de valor acrescentado para oferecer uma visão integrada da ciência no país.

Apesar do papel fundamental do *PTCRIS* na normalização e integração dos sistemas de informação científica em Portugal, a plataforma que se está a desenvolver neste projeto distingue-se por se focar especificamente na interoperabilidade entre plataformas de currículos científicos, com ênfase na extração, transformação e exportação de dados para a plataforma *CIÊNCIAVITAE*.

Para dar resposta aos desafios da interoperabilidade entre sistemas, têm sido desenvolvidas diversas soluções tecnológicas que recorrem a APIs, formatos estruturados como *XML* (Extensible Markup Language) [6] ou *JSON* (JavaScript Object Notation) [7], e normas internacionais de descrição e partilha de dados [8]. Estas abordagens possibilitam a criação de sistemas intermediários capazes de transformar, validar e sincronizar dados entre plataformas distintas.

No entanto, a adoção e cobertura destas soluções continua a ser desigual, e muitas pla-

taformas relevantes ainda não se encontram integradas entre si, o que mantém a necessidade de intervenções manuais e limita a eficiência dos processos de gestão curricular.

2.2 Sistemas semelhantes

A *CIÊNCIAVITAE* [1] é uma plataforma nacional portuguesa, promovida pela Fundação para a Ciência e a Tecnologia (FCT) [9], que permite a gestão integrada, personalizada e contínua de currículos científicos. Tem como principal objetivo centralizar a produção científica, académica e profissional num único perfil digital, promovendo a interoperabilidade com outros sistemas nacionais e internacionais.

Atualmente, a *CIÊNCIAVITAE* permite a importação de dados a partir de várias fontes, nomeadamente:

- *ORCiD* [4] - identificador internacional de autores;
- *SciPROJ* [10] - gestão de projetos de investigação e desenvolvimento;
- *RCAAP* [11] - repositórios de publicações científicas;
- *PRIES* [12] - identificadores de instituições científicas;
- *RENATES* [13] - base nacional de teses e dissertações;

Apesar destas integrações, a *CIÊNCIAVITAE* ainda não suporta a importação direta de dados provenientes de outras plataformas relevantes, como o *Lattes*, amplamente utilizado no Brasil. Esta limitação cria obstáculos à interoperabilidade internacional, obrigando os utilizadores a duplicar o trabalho de introdução de dados ou a recorrer a soluções manuais, sujeitas a erros e inconsistências.

O sistema *Lattes* [2], mantido pelo *Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq)* [14], é uma referência consolidada na gestão de currículos científicos no Brasil, oferecendo exportação em formato *XML*. No entanto, não existe, atualmente, um mecanismo oficial que permita a sua integração automática com a *CIÊNCIAVITAE*.

Capítulo 3

Arquitetura

Este capítulo irá focar-se na arquitetura da aplicação, incluindo os seus componentes, os casos de uso identificados para o sistema e o modelo de dados.

3.1 Diagrama de Blocos

O diagrama de blocos de alto nível da aplicação é representado na Figura 3.1

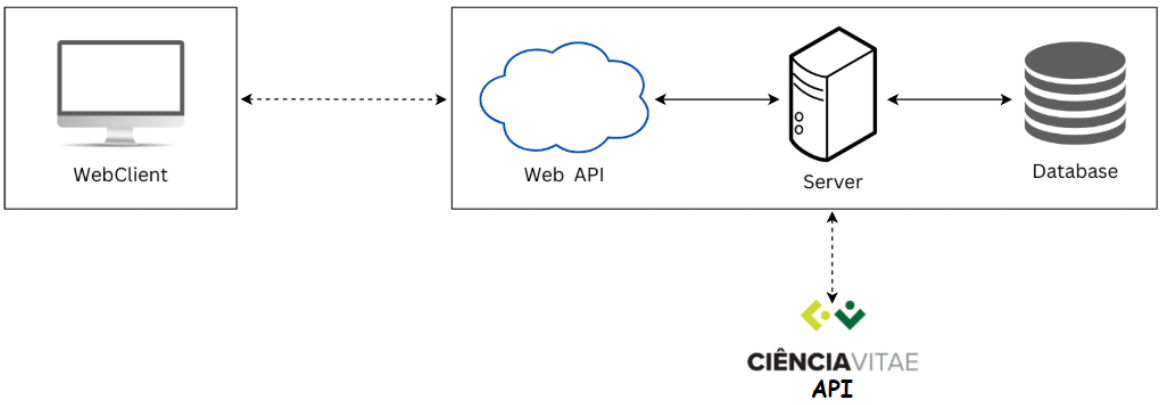


Figura 3.1: Diagrama de blocos de alto nível da aplicação

A arquitetura da aplicação é composta por três camadas principais: cliente, servidor e base de dados, conforme ilustrado na Figura 3.1. Esta estrutura modular assegura uma separação clara de responsabilidades e facilita a manutenção.

1. **WebClient**: Componente responsável pela interface com o utilizador. Esta camada é implementada como uma aplicação *web*, permitindo a visualização, edição e submissão de currículos científicos de forma intuitiva.

2. **Web API**: Camada de comunicação entre o cliente e o servidor, baseada em *HTTP*. A *Web API* expõe os endpoints necessários para as funcionalidades da aplicação.

3. **Server**: Centraliza o processamento principal da aplicação. Aqui são tratados os pedidos recebidos via *Web API*, aplicadas regras e validações, e coordenada a persistência de dados. É também nesta camada que se processam os dados importados, se realizam as transformações necessárias e se prepara a estrutura de dados compatível com a *CIÊNCIAVITAE*.

4. **Database**: Sistema de base de dados relacional, responsável pelo armazenamento estruturado da informação, incluindo dados de utilizadores, sessões e currículos científicos.

5. **Integração com a CIÊNCIAVITAE**: O sistema utiliza a API oficial da plataforma *CIÊNCIAVITAE* [3] para realizar a troca direta de dados do currículo com o perfil do utilizador na plataforma.

3.2 Casos de Uso

Com base nos requisitos definidos para a plataforma, foram identificados os principais casos de uso que o sistema deverá suportar. Um caso de uso consiste numa descrição de como os utilizadores irão interagir com o sistema para realizar determinadas tarefas.

Os casos de uso identificados permitem compreender as ações que estão disponíveis aos utilizadores da plataforma e a forma como o sistema deverá reagir a cada uma dessas ações. Para melhor organização e clareza, os casos de uso foram divididos em dois grupos principais, conforme apresentado nas Figuras 3.2 e 3.3:

1. **Casos de Uso de Autenticação:** relacionados com a autenticação e gestão da conta do utilizador.

2. **Casos de Uso de Gestão de Currículo:** relacionados com a importação, edição e exportação do currículo científico (CV).

3.2.1 Casos de Uso de Autenticação

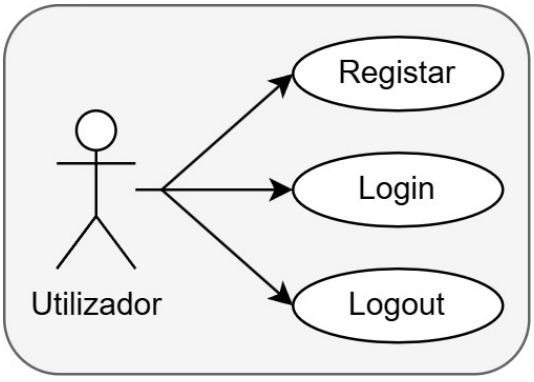


Figura 3.2: Casos de Uso de Autenticação

O primeiro conjunto de casos de uso, representado na Figura 3.2, apresenta as ações essenciais que o utilizador pode realizar relativamente à sua conta na plataforma. Estas ações estão disponíveis assim que o utilizador acede à plataforma e incluem:

- **Registar:** Permite ao utilizador criar uma nova conta para aceder aos serviços da plataforma, fornecendo os dados necessários para a sua identificação e autenticação.
- **Login:** Após o registo, o utilizador pode autenticar-se na plataforma usando as suas credenciais para iniciar uma sessão segura.
- **Logout:** Quando o utilizador termina a sua sessão, pode efetuar *logout* para garantir a segurança e proteger os seus dados.

Estes casos de uso representam as interações fundamentais para garantir o acesso controlado e personalizado à plataforma.

3.2.2 Casos de Uso de Gestão de Currículo

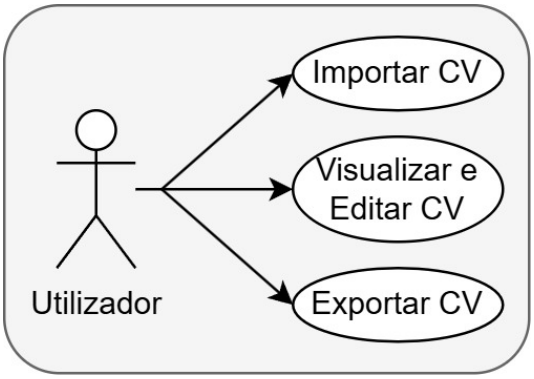


Figura 3.3: Casos de Uso de Gestão de Currículo

Após autenticação, o utilizador pode aceder às funcionalidades específicas relacionadas com a gestão do seu currículo científico. Os casos de uso deste grupo, representado na Figura 3.3 incluem:

- **Importar CV:** Permite ao utilizador importar o seu currículo científico a partir de uma plataforma externa diferente da plataforma CIÊNCIAVITAE. O sistema suporta a leitura e o processamento dos dados para posterior armazenamento.
- **Visualizar e Editar CV:** Após a importação, o utilizador pode consultar o conteúdo do seu currículo científico e, caso necessário, atualizar ou corrigir informações, garantindo que os dados refletidos na plataforma estão atualizados.
- **Exportar CV:** Permite ao utilizador exportar o seu currículo científico para a plataforma *CIÊNCIAVITAE*. O sistema utiliza um modelo interno de CV (ver Secção 3.3),

que serve como formato intermédio, garantindo a transformação e estruturação dos dados conforme os requisitos da plataforma de destino.

3.3 Modelo de Dados

A Figura 3.4 apresenta uma versão simplificada do diagrama Entidade–Associação (EA), com foco nas entidades e relações centrais. Esta figura é utilizada para contextualizar o modelo no corpo do relatório, enquanto o diagrama completo, com todos os atributos e entidades auxiliares, pode ser consultado no Apêndice A, na Figura A.1. Este modelo representa a estrutura de dados do sistema de gestão de currículos desenvolvido neste projeto.

Embora tenha sido desenhado com atenção à compatibilidade com o modelo de dados da plataforma *CIÊNCIAVITAE* [3], a sua estrutura é genérica para representar currículos científicos de forma independente de qualquer plataforma específica. O modelo orienta-se por princípios de normalização e modularidade, de forma a garantir a integridade dos dados.

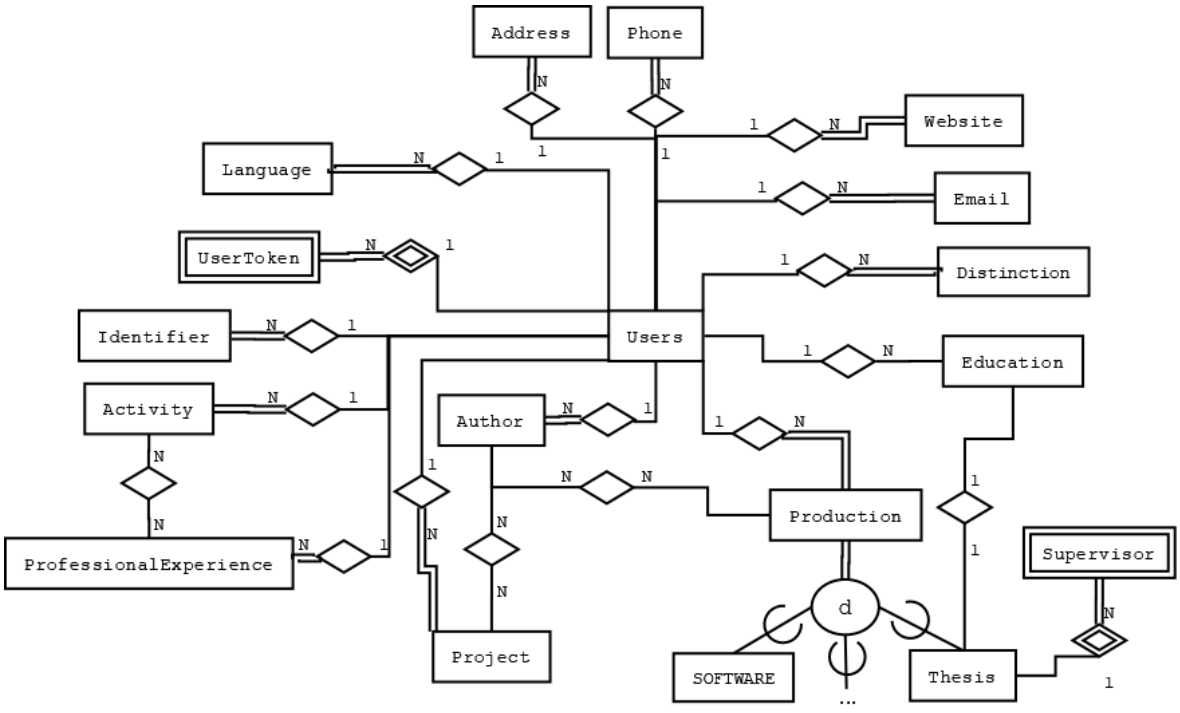


Figura 3.4: Visão simplificada do diagrama EA com as principais entidades.

Estrutura Central: Utilizadores e Currículos

A entidade **Users** representa os utilizadores do sistema, sendo o ponto de entrada para toda a informação curricular. Cada utilizador possui atributos como *name*, *email*, *password* e *cienciaID*, este último permitindo a interoperabilidade com plataformas externas como a CIÊNCIAVITAE. A autenticação é gerida pela entidade fraca associada **UserToken**, que armazena o *token* de sessão, bem como as datas de emissão e expiração.

Cada utilizador pode possuir múltiplos elementos curriculares associados, o que motivou a modelação de entidades específicas para cada uma das dimensões do currículo, em vez de utilizar estruturas genéricas.

Produções

Uma das partes mais críticas do modelo é a entidade **Production**, que representa produções científicas. Esta entidade foi desenhada com base numa estrutura de generalização, permitindo a especialização por tipo de produção: *ARTICLE*, *BOOK*, *EDITED_BOOK*, *REPORT*, *SOFTWARE*, *WEBSITE*, *RESEARCH_TECHNIQUE*, *PATENT*, *OTHER*, *THESIS*.

Cada subentidade possui atributos específicos (como *issn*, *isbn*, *url*, *doi*, *volume*, etc.), de forma a capturar os detalhes relevantes para cada tipo de produção. Esta abordagem oferece elevada flexibilidade e permite representar tanto produções contemporâneas altamente formalizadas, como artigos com DOI, como produções mais antigas ou não indexadas, que podem não possuir qualquer identificador.

Assim, decidiu-se modelar todas as informações relevantes da produção (título, autores, data de publicação, etc.) diretamente nas entidades, em vez de depender exclusivamente de identificadores externos como *DOI*, *ISSN* ou *ISBN*. Isto porque:

- Nem todas as produções científicas possuem identificadores formais, especialmente produções mais antigas ou informais (ex.: software experimental, pré-publicações, relatórios internos).
- A captura dos metadados completos no modelo garante maior independência face a fontes externas.

A entidade **Author** estabelece uma relação muitos-para-muitos com a entidade **Production**, permitindo listar todos os autores envolvidos em cada produção científica. Além disso, a entidade **Author** está associada à entidade **User** para representar os nomes de citação usados pelos utilizadores na plataforma. Essa associação é importante para garantir que os nomes dos autores nas produções científicas reflitam corretamente as formas preferidas de citação de cada utilizador, facilitando a identificação precisa.

Experiência e Atividades

A entidade **ProfessionalExperience** permite descrever cargos, funções ou atividades profissionais exercidas ao longo do percurso do utilizador, incluindo descrição, datas de início e fim, e função desempenhada. Para melhor refletir a diversidade de experiências, esta entidade contempla especializações específicas, tais como:

- *Science* — atividades de pesquisa científica;
- *Teaching_HE* — ensino superior;

- *Positions* — cargos administrativos ou funções representativas;
- *Others* — outras experiências profissionais relevantes que não se enquadram nas categorias anteriores.

A entidade genérica **Activity** foi introduzida para registar uma variedade de atividades profissionais e académicas que não se encaixam diretamente nos conceitos anteriores. Esta entidade contempla eventos como organização de eventos, consultorias, participação em bancas académicas, disciplinas ministradas, supervisão académica, entre outros. A estrutura do modelo contempla uma tabela base *Activity* com atributos comuns (*id*, *title*, *date*) e diversas tabelas especializadas que herdam dessa entidade, representando cada tipo específico de atividade, tais como:

- *Supervision*, que regista atividades de supervisão académica com informações detalhadas sobre o papel, supervisionado, instituição, curso, código do curso e grau;
- *Subject_Taught*, referente a disciplinas ministradas, com dados do curso e datas;
- *Academic_Jury*, para a participação em bancas académicas, com dados do candidato, instituição, curso e grau;
- *Event*, para eventos organizados, incluindo data final;
- *Consulting*, para atividades de consultoria, com a instituição correspondente.

Esta modelagem detalhada permite capturar a multiplicidade de experiências e atividades que compõem o perfil académico e profissional do utilizador. A ligação entre atividades e experiências profissionais é assegurada pela tabela associativa *Activity_ProfessionalExperience*, que possibilita relacionar cada atividade a uma ou várias experiências profissionais e vice-versa.

Educação e Projetos

As entidades **Education** e **Project** cobrem duas áreas fundamentais do percurso académico e profissional do utilizador.

A entidade **Education** armazena os dados de formações concluídas ou em curso, com atributos como *degree*, *institution*, *startDate*, *endDate*, *courseCode* e *status*. Esta entidade está associada ao tipo de produção **Thesis**, que permite registar uma tese — quando aplicável — e os respetivos orientadores, através da ligação à entidade **Supervisor**, onde é possível indicar o nome e o papel (orientador, coorientador, etc.) de cada um.

A entidade **Project** representa a participação em projetos científicos ou académicos, com dados como título, instituição, função desempenhada, tipo de financiamento, datas e identificador. Um projeto pode ter vários autores que, por sua vez, podem participar em vários projetos.

Idiomas, Identificadores e Contactos

A entidade **Language** representa o conhecimento linguístico do utilizador, com quatro níveis de competência (compreensão, leitura, escrita e oralidade).

A entidade **Identifier** permite associar identificadores externos como *ORCID*, *ResearcherID* ou *Scopus Author ID*, promovendo a integração com outras bases de dados.

A modelação dos contactos foi dividida em quatro entidades específicas: **Phone**, **Email**, **Website** e **Address**, permitindo a representação estruturada e normalizada de cada tipo de meio de contacto.

Restrições de Integridade

Para garantir a consistência e integridade dos dados no modelo apresentado, foram definidas diversas restrições de integridade, detalhadas na Tabela A.1. Esta tabela apresenta as restrições de integridade, validação e consistência aplicadas às entidades e seus atributos no modelo relacional.

Uso Interno de DTOs e Agregação Lógica

Apesar da separação lógica e normalizada das entidades no modelo de dados, foi utilizada uma abordagem prática na implementação do sistema *backend* através de **Data Transfer Objects** (DTOs), que agrupam os dados para facilitar a comunicação entre camadas. O exemplo mais representativo é o **CvDTO**, que encapsula todas as informações do currículo de um utilizador (incluindo listas de produções, idiomas, formações, projetos, etc.) num único objeto agregado.

Esta abstração:

- Simplifica a serialização e deserialização de dados para troca via API REST.
- Facilita a exportação e importação de currículos como blocos coesos.
- Ajuda a manter uma separação clara entre o modelo persistente (base de dados) e o modelo de apresentação/integração (DTOs).

Considerações Finais

O modelo Entidade–Associação (EA) desenvolvido constitui a base estrutural do sistema de gestão de currículos, assegurando uma organização clara e modular da informação. A sua conceção teve em vista a flexibilidade e extensibilidade, permitindo representar com detalhe diversas dimensões da trajetória académica e profissional dos utilizadores.

A conversão deste modelo conceptual para o modelo relacional foi realizada com o objetivo de preservar a integridade lógica das entidades e das associações definidas. Foram consideradas boas práticas de normalização, evitando redundâncias e assegurando a consistência dos dados ao longo do ciclo de vida da aplicação.

Este modelo fornece, assim, uma fundação sólida e fiável para a implementação das funcionalidades do sistema, facilitando tanto o armazenamento eficiente como a recuperação estruturada da informação. Além disso, a sua modularidade contribui para a evolução futura da aplicação, permitindo incorporar novos tipos de dados ou funcionalidades sem comprometer a integridade do sistema existente.

Capítulo 4

Tecnologias

Neste capítulo, são descritas as principais tecnologias que sustentam os diversos componentes do projeto, com uma explicação sobre o seu funcionamento e as razões que justificam a sua adoção no contexto deste sistema.

Inicialmente, são abordadas as tecnologias de *backend*, seguidas pelas de *frontend* e, por fim, as ferramentas que apoiaram o desenvolvimento, testes e documentação do sistema.

Grande parte das tecnologias aqui abordadas foi adquirida e aprofundada ao longo do percurso académico, influenciando significativamente as decisões técnicas tomadas ao longo do desenvolvimento deste projeto.

4.1 *Backend*

As tecnologias de *backend* utilizadas no projeto desempenham um papel fundamental na funcionalidade do sistema. Estas tecnologias englobam os componentes do lado do servidor responsáveis pelo processamento de dados, gestão da base de dados e integração com a *API* da plataforma *CIÊNCIAVITAE* [3].

Nesta secção, será apresentada uma análise detalhada das principais ferramentas adotadas no desenvolvimento do *backend*, destacando as suas funcionalidades, vantagens e a forma como colaboram para assegurar o funcionamento eficiente e o bom desempenho da aplicação.

4.1.1 Kotlin

Kotlin [15] foi a linguagem escolhida para o desenvolvimento do *backend* devido à sua modernidade, expressividade e segurança. Compilando para *bytecode Java*, é totalmente interoperável com o ecossistema *Java* [16], o que permite integrar facilmente bibliotecas e *frameworks* já consolidados, como o *Spring* [17]. A sua sintaxe concisa e clara reduz significativamente a verbosidade do código, contribuindo para uma maior produtividade e facilidade de manutenção.

A escolha por *Kotlin* foi também motivada pela familiaridade prévia com a linguagem, o que permitiu acelerar o processo de desenvolvimento e aplicar boas práticas desde o início

do projeto. A sua forte integração com o *Spring* — incluindo suporte nativo em projetos com *Spring Boot* — tornou o *Kotlin* particularmente adequado para a construção da *API*, reduzindo a complexidade na implementação dos serviços.

4.1.2 Spring

O projeto utiliza o *framework Spring*, com destaque para o *Spring Boot*, para acelerar o desenvolvimento da aplicação *backend*. A escolha por *Spring* deve-se à sua capacidade de simplificar a criação de APIs *RESTful*, integrar serviços externos e aplicar boas práticas de segurança com o mínimo de configuração manual.

Com o *Spring Boot* [18], é possível configurar e executar a aplicação de forma simples e rápida, já que o ambiente de execução é gerido diretamente pelo próprio projeto. A criação de *endpoints REST* (Representational State Transfer) é direta e eficiente, com suporte nativo à serialização de dados em *JSON* [7] e *XML* [6] — essencial para a compatibilidade com a plataforma *CIÊNCIAVITAE* [1].

Entre os componentes do ecossistema *Spring* utilizados neste projeto, destacam-se:

- **BCryptPasswordEncoder:** [19] para codificação segura das palavras-passe, garantindo proteção contra ataques.
- **WebClient:** [20] cliente *HTTP* (HyperText Transfer Protocol) usado para comunicação assíncrona com a *API* da *CIÊNCIAVITAE*, suportando envio e receção de dados em *XML*.

A familiaridade com o *framework*, a vasta documentação e a forte comunidade de suporte também contribuíram para a escolha do *Spring* como base para o desenvolvimento desta aplicação.

4.1.3 Jackson XML

A biblioteca *Jackson XML* [21] é utilizada para a serialização e desserialização de documentos *XML* trocados com a plataforma *CIÊNCIAVITAE*, bem como para o processamento de dados importados da plataforma *Lattes*.

Esta biblioteca permite transformar estruturas *XML* em objetos *Kotlin (DTOs)* e vice-versa, facilitando a integração entre os serviços internos e os *endpoints* externos que exigem dados estruturados em *XML*.

A escolha por *Jackson* deve-se à sua integração nativa com *Spring* e ao suporte robusto a anotações personalizadas, permitindo lidar com formatos *XML* complexos.

4.1.4 PostgreSQL

A aplicação utiliza *PostgreSQL* [22] como sistema de gestão de base de dados relacional, escolhido pela sua compatibilidade com o ecossistema *Spring* e bom suporte a tipos de dados

estruturados.

A persistência dos dados é feita através do *Spring Data JPA* [23], que permite interagir com a base de dados de forma declarativa, utilizando repositórios *Kotlin* que abstraem grande parte do código *SQL*. Esta camada de persistência armazena dados dos utilizadores e currículos científicos.

4.2 Frontend Web

No desenvolvimento do *frontend*, foram utilizadas tecnologias que fornecem uma interface visualmente apelativa e fácil de usar, permitindo aos utilizadores interagir eficazmente com o sistema.

Estas tecnologias englobam os componentes do lado do cliente responsáveis pela renderização da interface, pelo processamento das interações dos utilizadores e pela garantia de uma experiência fluida e responsiva.

Nesta secção, será feita uma análise detalhada das principais ferramentas que foram utilizadas no desenvolvimento da interface, evidenciando como estas contribuem para a criação de uma experiência de utilizador intuitiva e acessível.

4.2.1 TypeScript

TypeScript [24] adiciona tipagem estática ao desenvolvimento, facilitando a deteção precoce de erros em tempo de compilação e tornando a manutenção de aplicações complexas mais eficiente. Esta linguagem suporta as funcionalidades mais recentes do *JavaScript* [25], como classes, interfaces e módulos.

A escolha pelo *TypeScript* baseia-se na sua escalabilidade e forte integração com ferramentas modernas, tornando-o especialmente adequado para projetos de maior dimensão e para assegurar a qualidade do código no desenvolvimento *frontend*.

4.2.2 React

React [26] é uma biblioteca *JavaScript* baseada em componentes, amplamente adotada para a criação de interfaces de utilizador dinâmicas e reutilizáveis. Permite o desenvolvimento de aplicações *single-page*, proporcionando uma experiência de navegação rápida e fluida.

No projeto, o *React* será utilizado para estruturar a interface através de componentes modulares, o que facilita tanto a reutilização quanto a manutenção do código.

Para gerir a navegação entre as diferentes secções da aplicação, será usada a biblioteca *React Router* [27], que assegura uma transição eficiente e preserva o estado da aplicação.

4.3 Deployment

O processo de *deployment* da aplicação utiliza *containers* para facilitar a instalação e execução do sistema em diferentes ambientes, garantindo que todos os componentes funcionem corretamente e da mesma forma em qualquer máquina.

4.3.1 Docker

O *Docker* [28] foi utilizado para empacotar os principais componentes do sistema — *frontend web*, *backend* e base de dados — em *containers* isolados. Cada serviço possui a sua própria imagem e ambiente de execução, o que elimina dependências externas e reduz os erros de configuração entre diferentes máquinas.

Esta abordagem garante que a aplicação pode ser executada de forma consistente, independentemente do sistema operativo ou ambiente de desenvolvimento.

4.3.2 Docker Compose

Para facilitar a orquestração dos serviços, foi utilizado o *Docker Compose* [29]. Através de um ficheiro `docker-compose.yml`, são definidas as configurações de rede, volumes e variáveis de ambiente, permitindo iniciar todos os componentes com um único comando.

4.4 Suporte ao Desenvolvimento

Nesta secção, são descritas as tecnologias utilizadas no suporte ao ciclo de vida da aplicação, alinhadas com boas práticas de desenvolvimento.

Foram adotadas ferramentas que promovem uma gestão eficiente do código, a validação das funcionalidades e a documentação técnica do sistema. Estas ferramentas contribuíram significativamente para a organização do desenvolvimento, a deteção precoce de erros e a clareza na representação da arquitetura e dos processos do sistema.

4.4.1 Git e GitHub

Git [30] foi utilizado para controlo de versões durante todo o desenvolvimento, permitindo gerir as alterações do código de forma organizada e segura.

O repositório foi alojado no *GitHub* [31], que serviu como plataforma para armazenamento remoto e *backup* do projeto, facilitando o acesso e a gestão do código ao longo do tempo.

4.4.2 Postman

Postman [32] foi adotado como ferramenta principal para o teste e validação da *API* desenvolvida. Através de uma interface intuitiva, foi possível simular diversas chamadas *HTTP*, testar *endpoints* com diferentes *payloads* e analisar as respostas recebidas.

Esta prática assegurou que a comunicação entre os componentes do sistema funcionasse conforme esperado, permitindo detetar e corrigir falhas precocemente.

4.4.3 Dia e diagrams.net

As ferramentas *Dia* [33] e *diagrams.net* [34] foram utilizadas para a criação de diagramas técnicos, incluindo diagramas Entidade-Associação (EA), fluxogramas e documentação visual do sistema.

Estas ferramentas permitiram representar de forma clara a arquitetura, os fluxos de dados e os processos envolvidos no projeto, facilitando o planeamento, a comunicação e a apresentação do desenvolvimento.

Capítulo 5

Desenvolvimento

Neste capítulo, será descrita a abordagem de desenvolvimento, a estrutura do projeto e os principais aspetos técnicos do *backend* da aplicação, tais como o acesso à base de dados, chamadas a serviços externos, autenticação de utilizadores, gestão de sessões, *parsing* e persistência de currículos, entre outros. Será também abordada a implementação do *frontend*, que permite a interação direta dos utilizadores com o sistema através de uma interface *web*.

5.1 Metodologia de Desenvolvimento

A implementação do sistema seguiu uma abordagem tradicional baseada no desenvolvimento por camadas completas, em que cada camada da aplicação (base de dados, domínio, repositórios, serviços e controladores) foi construída de forma sequencial e coesa para cada conjunto de funcionalidades.

A primeira funcionalidade desenvolvida foi o sistema de autenticação, que inclui o registo, *login* e *logout* de utilizadores. Após garantir uma base segura para o acesso dos utilizadores, passou-se para o desenvolvimento da funcionalidade de importação de currículos. Posteriormente, foi implementada a funcionalidade de exportação de currículos para a plataforma *CIÊNCIAVITAE*, concluindo assim as principais funcionalidades da *API*. Por fim, foi desenvolvida a interface *web*, que permite a interação direta dos utilizadores com a aplicação.

Esta abordagem por camadas completas permitiu garantir que todas as dependências internas estivessem bem definidas e testadas antes de passar para o desenvolvimento da camada seguinte. Além disso, promoveu maior reutilização de código e modularidade ao longo do projeto.

5.2 Estrutura do Projeto

A estrutura do projeto foi desenhada seguindo princípios de separação de responsabilidades, organização em camadas e facilidade de manutenção.

5.2.1 Estrutura de Diretórios

A Figura 5.1 apresenta a estrutura de diretórios do projeto, agora organizada em duas partes principais: **backend**, que segue o padrão *MVC (Model-View-Controller)* adaptado ao ecossistema *Spring Boot*, e **frontend**, que implementa a camada de apresentação com *React* e *TypeScript*.

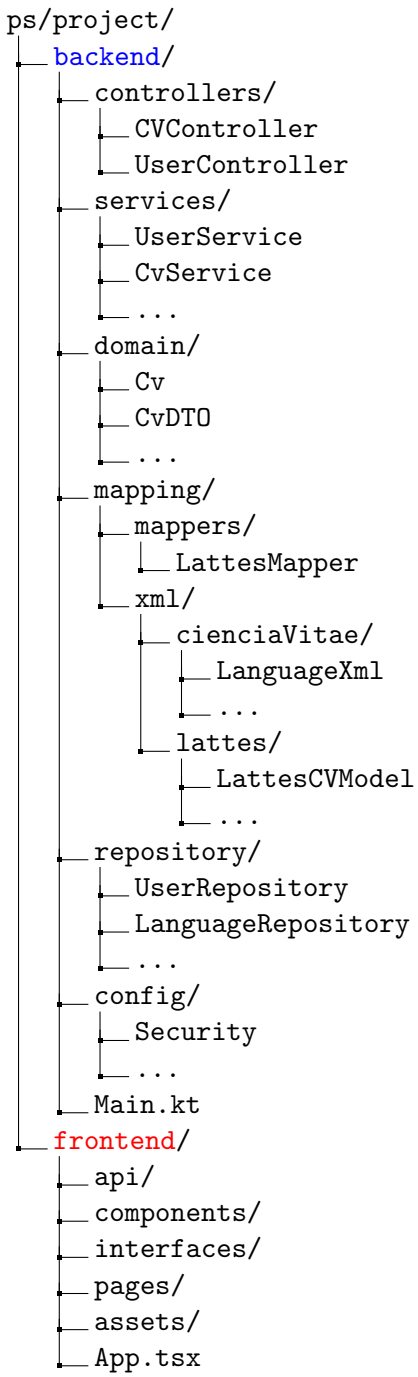


Figura 5.1: Estrutura de diretórios do projeto

Cada camada tem uma função clara dentro do sistema:

- **backend/**: Contém a aplicação *Spring Boot* organizada segundo o padrão *MVC*.
  - **controllers/**: Contém os controladores *REST* responsáveis por receber requisições *HTTP* e encaminhá-las para os serviços apropriados.
  - **services/**: Implementam a lógica da aplicação, subdividida em serviços específicos.
  - **domain/**: Contém as entidades de domínio e *DTOs*. Estas classes representam os dados tratados pela aplicação.
  - **mapping/**: Responsável pela conversão entre estruturas *XML* e objetos de domínio. Está organizado em:
    - *mappers/* – contém os mapeadores responsáveis por converter os dados de diferentes fontes para os *DTOs* internos do sistema. Inclui, por exemplo, o *LattesMapper*, que trata especificamente do formato *Lattes*.
    - *xml/* – contém as classes de suporte para deserialização com *Jackson*, subdivididas em *lattes/* e *cienciaVitae/*.
  - **repository/**: Interface com a base de dados, composta por repositórios *Spring Data*.
  - **config/**: Contém as configurações da aplicação.
- **frontend/**: Implementa a camada de apresentação da aplicação utilizando *React* e *TypeScript*.
  - **api/**: Contém funções e serviços para comunicação com o *backend*.
  - **components/**: Componentes reutilizáveis da interface de utilizador (*UI*).
  - **interfaces/**: Tipagens *TypeScript* utilizadas para estruturar os dados manipulados pela aplicação *frontend*.
  - **pages/**: Páginas principais da aplicação, cada uma representando uma rota no sistema.
  - **assets/**: Contém os *assets* da aplicação, como imagens e ícones;

5.2.2 Endpoints REST

A Tabela 5.1 apresenta os principais *endpoints* expostos pela *API REST* da aplicação. Todos os caminhos estão aninhados sob o prefixo */api*. Estes *endpoints* permitem as funcionalidades principais do sistema.

Endpoint	**Método HTTP**	**Descrição**	**Autenticação**
/users/register	POST	Registo de novos utilizadores	Não
/users/login	POST	Autenticação de utilizadores	Não
/users/logout	POST	*Logout* e invalidação do *token*	Sim
/cv	GET	Obtenção dos dados do CV	Sim
/cv/import	POST	Importação de dados do CV	Sim
/cv/send	POST	Exportação do CV para a CIÊNCIAVITAE	Sim
/cv/edit	PUT	Edição de dados do CV	Sim

Tabela 5.1: Principais *endpoints* da API REST e necessidade de autenticação

5.3 Autenticação e Gestão de Utilizadores

A autenticação dos utilizadores é realizada com base em *tokens UUID* gerados no momento do *login*, adotando uma abordagem *stateless* com *Spring Security*. O controlo de acesso é garantido por um filtro personalizado, que valida os *tokens* presentes no cabeçalho *Authorization*.

5.3.1 Registo de Utilizadores

O processo de registo é feito através do *endpoint POST /api/users/register*, descrito na Tabela 5.1, e inclui:

- Verifica se o email inserido já existe na base de dados;
- Codificação da palavra-passe com *BCryptPasswordEncoder*;
- Obtenção do nome do utilizador a partir do identificador *cienciaID*, via *API* da *CIÊNCIAVITAE*;
- Persistência da entidade *User*.

5.3.2 Login de Utilizadores

O *login* é realizado em *POST /api/users/login*, descrito na Tabela 5.1, e retorna um *token UUID* com validade de 24 horas, associado ao utilizador na base de dados. O processo da geração do *token* está representado na Figura 5.2.

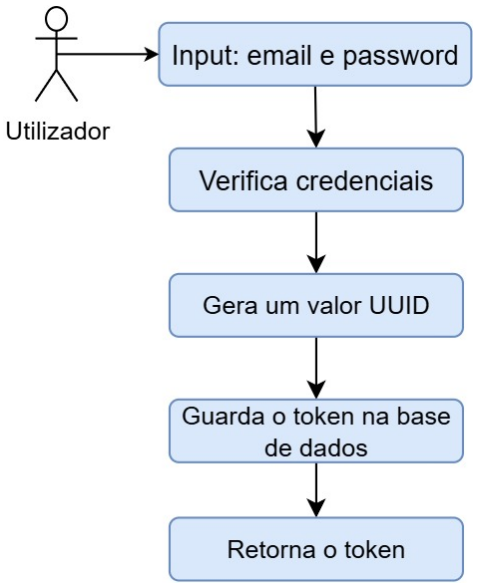


Figura 5.2: Geração do *token* de autenticação

5.3.3 Logout de Utilizadores

O *logout* é realizado através do *endpoint POST /api/users/logout* descrito na Tabela 5.1, que remove o *token* da base de dados, invalidando a sessão do utilizador.

Esta operação é transacional e garante que os *tokens* removidos não são mais aceites.

5.3.4 Filtro de Autenticação

O componente *TokenAuthenticationFilter* é responsável por:

1. Extrair o *token* do cabeçalho *Authorization*;
2. Validar a existência e validade do *token* na base de dados;
3. Definir o contexto de autenticação do utilizador, armazenando a informação do utilizador autenticado no *SecurityContext*, uma estrutura do *Spring Security* que mantém os dados de autenticação durante toda a requisição. Isso permite que outras partes do sistema saibam quem é o utilizador atual e apliquem as permissões adequadas.

Este filtro é registado na cadeia de segurança do *Spring Security*, antes do filtro padrão de autenticação por nome de utilizador e *password*, conhecido como *UsernamePasswordAuthenticationFilter*. Este filtro padrão é utilizado em aplicações que fazem *login* tradicional com nome de utilizador e palavra-passe em formulários. Como a aplicação utiliza apenas *tokens* para autenticação, o filtro personalizado assume essa responsabilidade.

O fluxo de autenticação baseado em *token* encontra-se representado na Figura 5.3.

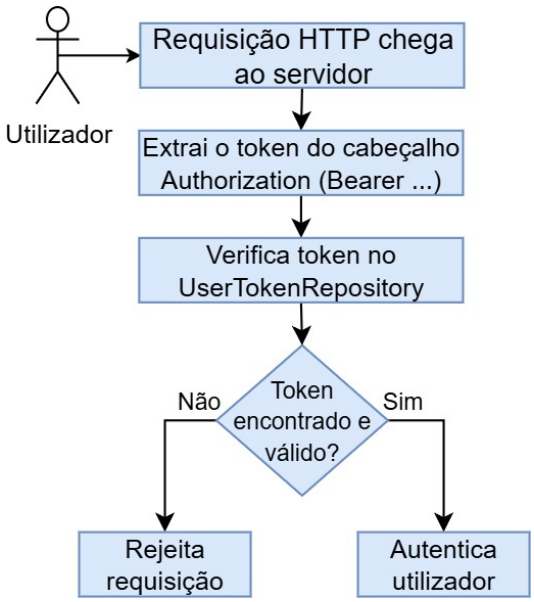


Figura 5.3: Fluxo do filtro de autenticação por *token*

5.3.5 Configuração de Segurança

A autenticação é obrigatória para aceder a rotas sensíveis, nomeadamente aquelas que manipulam dados curriculares ou sessões de utilizador.

Conforme apresentado na Tabela 5.1, os *endpoints* */api/users/logout*, */api/cv*, */api/cv/import*, */api/cv/send* e */api/cv/edit* requerem um *token* de autenticação válido.

Além disso, a gestão de sessões está desativada (*stateless*), reforçando o uso de *tokens* em vez de sessões de servidor.

5.4 Processamento e Integração de Currículos (CVs)

A funcionalidade de integração de currículos constitui o núcleo central deste sistema, tendo como principal objetivo permitir a importação, persistência e exportação estruturada de informações curriculares científicas. Para isso, o sistema foi concebido com uma arquitetura modular que combina componentes especializados em *parsing* XML, persistência em base de dados, serialização para formatos compatíveis com APIs externas e comunicação via HTTP.

5.4.1 Importação de CVs

A importação de currículos é um dos pontos centrais do sistema, permitindo integrar informações curriculares provenientes de diferentes plataformas. Como referido nos requisitos obrigatórios do projeto, na Subsecção 1.2.1, apenas foi implementada a importação do CV da plataforma *Lattes*.

O fluxo inicia-se através do *endpoint POST /api/cv/import* descrito na Tabela 5.1, que recebe dois parâmetros:

- Um ficheiro, via *MultipartFile* (ex: *XML*).
- Uma *source* (origem), que indica a fonte do currículo (ex: *"lattes"*).

A lógica de *parsing* está estruturada de forma extensível e modular. A responsabilidade de iniciar o processo de análise do currículo é delegada à interface *CvParser*, cuja função é verificar se a origem indicada é suportada e, em caso afirmativo, proceder à conversão do ficheiro para uma estrutura intermédia denominada *CvDTO*, utilizada internamente pelo sistema.

A principal implementação desta interface é o serviço *CvParserService*. Este serviço mantém uma lista de componentes especializados, designados *CvMapper*, aos quais delega a lógica específica de extração de dados consoante o formato do currículo. Cada componente *CvMapper* sabe interpretar currículos de uma origem específica (por exemplo, o *LattesMapper* para currículos da plataforma *Lattes*), convertendo o conteúdo *XML* para a estrutura *CvDTO*.

Para adicionar o suporte a uma nova fonte de dados, basta implementar a interface *CvMapper* numa nova classe que saiba interpretar o conteúdo da plataforma desejada. Esta nova classe deve ser anotada com *@Service*, o que garante a sua deteção automática pelo mecanismo de injeção de dependências do *Spring*. Desta forma, é automaticamente incluída na lista de mapeadores utilizados pelo *CvParserService*.

A Figura 5.4 apresenta o diagrama de classes associado a este processo.

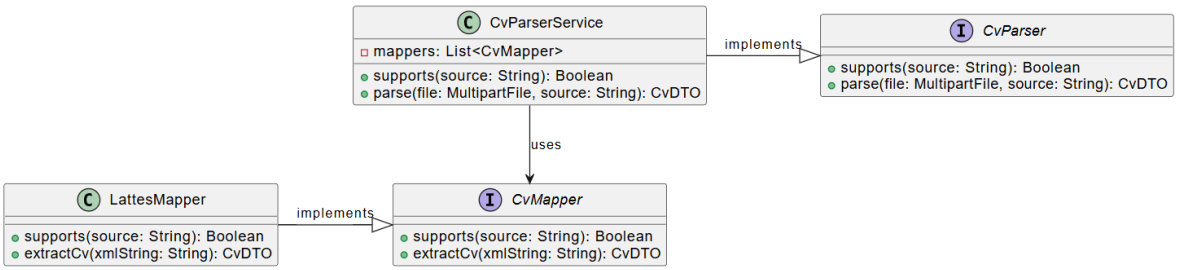


Figura 5.4: Diagrama de classes da arquitetura de importação de CVs

A interface *CvParser* é implementada pela classe *CvParserService*, enquanto a interface *CvMapper* é implementada por classes como *LattesMapper*. A dependência entre *CvParserService* e os *CvMapper* permite a introdução de novos formatos de currículo sem alterações no código existente, em conformidade com os princípios da responsabilidade única (SRP) e aberto/fechado (OCP) da programação orientada a objetos.

Resumo das vantagens da arquitetura adotada:

- **Modularidade** – Separação clara entre lógica de *parsing* genérica (*CvParserService*) e lógica específica por formato (*CvMapper*).

- **Extensibilidade** – Fácil inclusão de novos formatos apenas com novas implementações de *CvMapper*.

- **Testabilidade** – Cada *CvMapper* pode ser testado isoladamente, com *inputs* de ficheiros reais da respetiva plataforma.

- **Desacoplamento** – O *CvDTO* atua como uma camada de abstração entre o *parsing* e a lógica de persistência/exportação.

5.4.2 Implementação do *LattesCvMapper*

A plataforma Lattes fornece os currículos científicos em formato XML, com uma estrutura hierárquica detalhada que descreve dados pessoais, percurso académico, produções científicas, entre outros. Para processar estes ficheiros, foi desenvolvida uma implementação concreta da interface *CvMapper*, denominada *LattesMapper*, responsável por extrair e mapear os dados do XML para um objeto do tipo *CvDTO*, utilizado internamente pela aplicação.

Abordagem de mapeamento

A extração dos dados é realizada com a biblioteca *Jackson XML*, que permite deserializar diretamente o conteúdo do XML em classes Kotlin anotadas com metadados. Esta abordagem oferece uma maior legibilidade e reduz a propensão a erros face à manipulação manual com *parsers DOM* ou *SAX*, além de suportar facilmente valores nulos, listas não encadeadas e atributos do *XML*.

As classes Kotlin utilizadas no mapeamento são estruturadas de forma a refletir fielmente a organização hierárquica do *XML* da plataforma *Lattes*. Por exemplo, secções como "Idiomas", "Formação Académica" ou "Produção Bibliográfica" são modeladas como classes com propriedades anotadas para indicar correspondência direta com elementos *XML* e atributos. Estas são anotadas com anotações de *Jackson*, como *@JacksonXmlProperty* e *@JacksonXmlElementWrapper*.

Para listas de elementos repetidos (como várias entradas de idiomas), são usados *wrappers* que encapsulam essas listas e permitem que o *Jackson* trate corretamente a deserialização.

A Figura 5.5 ilustra um exemplo do processo que ocorre no mapeamento do XML do Lattes para o modelo interno do sistema, correspondente à secção de idiomas declarados pelo utilizador.

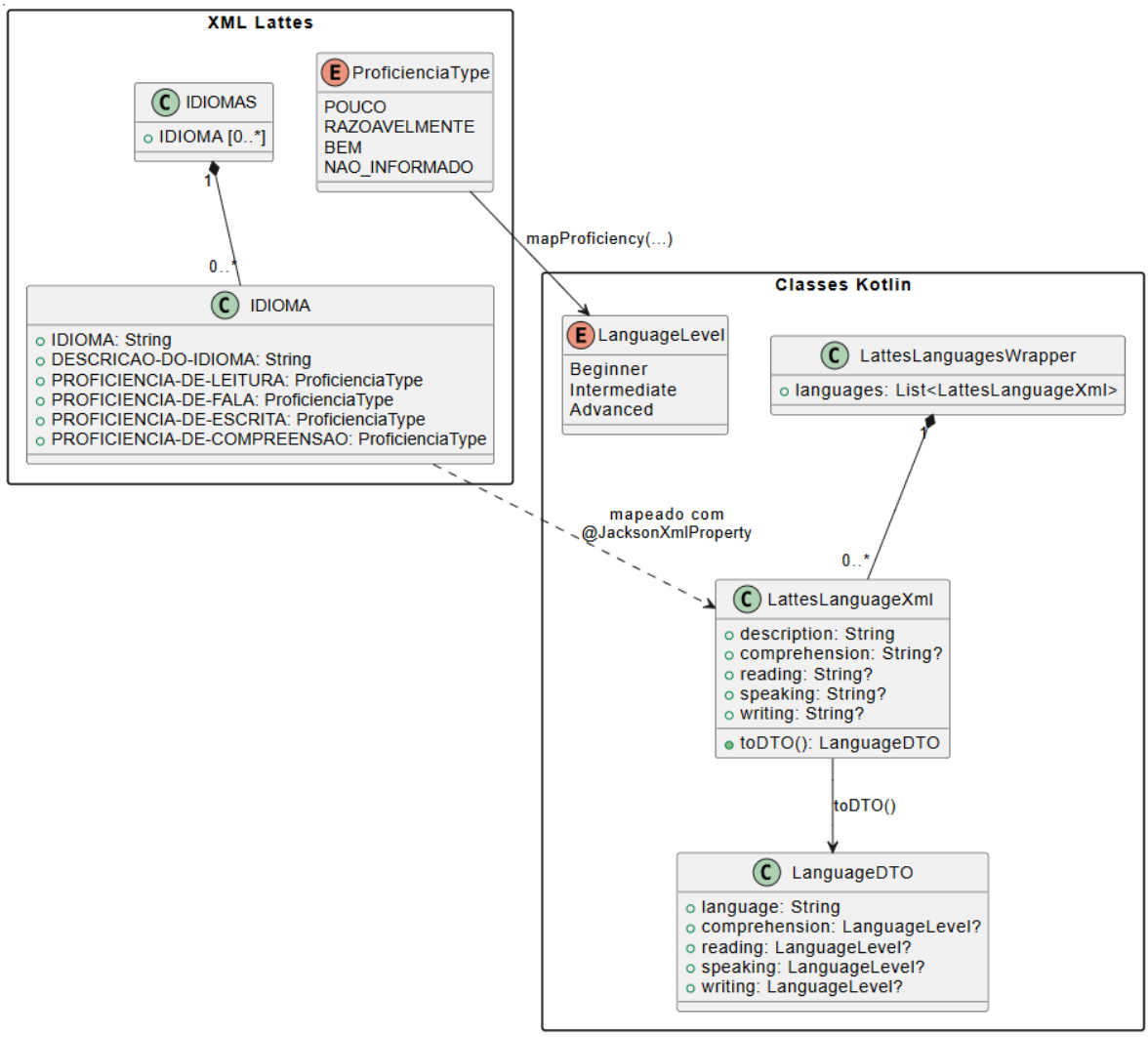


Figura 5.5: Exemplo do processo de mapeamento – Secção de Idiomas

Deserialização e processamento

Uma vez definidas as classes modelo, o conteúdo *XML* é processado por um objeto configurado de **XmlMapper**, o qual:

- Suporta a deserialização de elementos opcionais ou ausentes.
- Converte automaticamente valores vazios para *null*, quando apropriado.
- Ignora elementos desconhecidos no *XML*, apenas processando os elementos relevantes.

O resultado da deserialização é um objeto denominado *LattesCVModel*, que representa a totalidade do conteúdo do currículo *Lattes*. Este objeto utiliza internamente diversos tipos auxiliares — como *LattesLanguagesWrapperXML*, entre outros — que refletem a estrutura hierárquica do *XML* original. A partir do *LattesCVModel*, são extraídas de forma modular

as várias secções relevantes, como dados pessoais, produções científicas, experiência profissional, entre outras. Cada uma destas secções é então convertida para um *DTO* específico — por exemplo, idiomas são convertidos para objetos *LanguageDTO*, produções para *ProductionDTO*, e assim sucessivamente.

Esta abordagem permite isolar a complexidade do formato *XML* original e garantir que os dados estejam disponíveis numa estrutura interna normalizada, coesa e extensível.

5.4.3 Exportação para a plataforma *CIÊNCIAVITAE*

A funcionalidade de exportação é acessível através do *endpoint POST /api/cv/send* descrito na Tabela 5.1, permitindo que o utilizador inicie o processo de envio do currículo para a plataforma *CIÊNCIAVITAE*. Esta operação desencadeia um processo automático de transformação e envio dos dados curriculares, orquestrado pelo serviço *CvService*, que coordena a extração e formatação dos dados e delega o envio ao componente especializado *CvExportService*.

O objetivo principal desta funcionalidade é transformar e transmitir, de forma estruturada, os dados curriculares armazenados na aplicação, na forma do modelo interno de CV utilizado pela aplicação, para a API externa da *CIÊNCIAVITAE*. Esta exportação é feita através de chamadas *HTTP* com *payloads* em formato *XML* [6], respeitando os esquemas definidos pela própria plataforma.

Arquitetura do serviço de exportação:

A lógica de exportação está centralizada no serviço *CvExportService*, responsável por iterar sobre as diferentes secções do objeto *Cv* e enviá-las para os respetivos *endpoints* da *API* da *CIÊNCIAVITAE* [3]. A serialização dos dados em *XML* é realizada com a biblioteca *Jackson XML* [21], a qual permite gerar documentos compatíveis com os esquemas exigidos, incluindo atributos, *namespaces* e estrutura hierárquica.

A função *sendCvToCienciaVitae* é o ponto de entrada da exportação e é responsável por delegar o envio das diferentes secções do currículo. Cada tipo de informação possui uma função especializada que constrói o *payload XML* correspondente e envia-o utilizando a função genérica *postToCienciaVitae*.

Serialização em XML:

Cada secção do currículo é enviada separadamente, utilizando uma estrutura de *payload* comum baseada na classe abstrata *CienciaVitaeXmlPayload*, que encapsula metainformação como o nível de privacidade e a data da última modificação.

Esta abordagem permite garantir a consistência entre os diferentes tipos de *payload* e reduzir a duplicação de código, promovendo reutilização e simplificação na serialização de dados para os diferentes *endpoints* da API da plataforma.

Exemplo: Exportação de competências linguísticas: A exportação dos idiomas declarados pelo utilizador é realizada através da transformação de objetos do domínio interno (*Language*) em objetos compatíveis com o modelo *XML* da *CIÊNCIAVITAE* (*LanguageXml*). Estes objetos são então enviados para o *endpoint language-competency*.

A Figura 5.6 ilustra a arquitetura desta exportação. A classe *LanguageXml*, que herda de *CienciaVitaeXmlPayload*, representa a estrutura esperada no *XML*. Cada competência (ler, escrever, falar, compreender) é mapeada para elementos *XML* do tipo *CodeValueXml*, que por sua vez encapsula o código e valor do nível de proficiência.

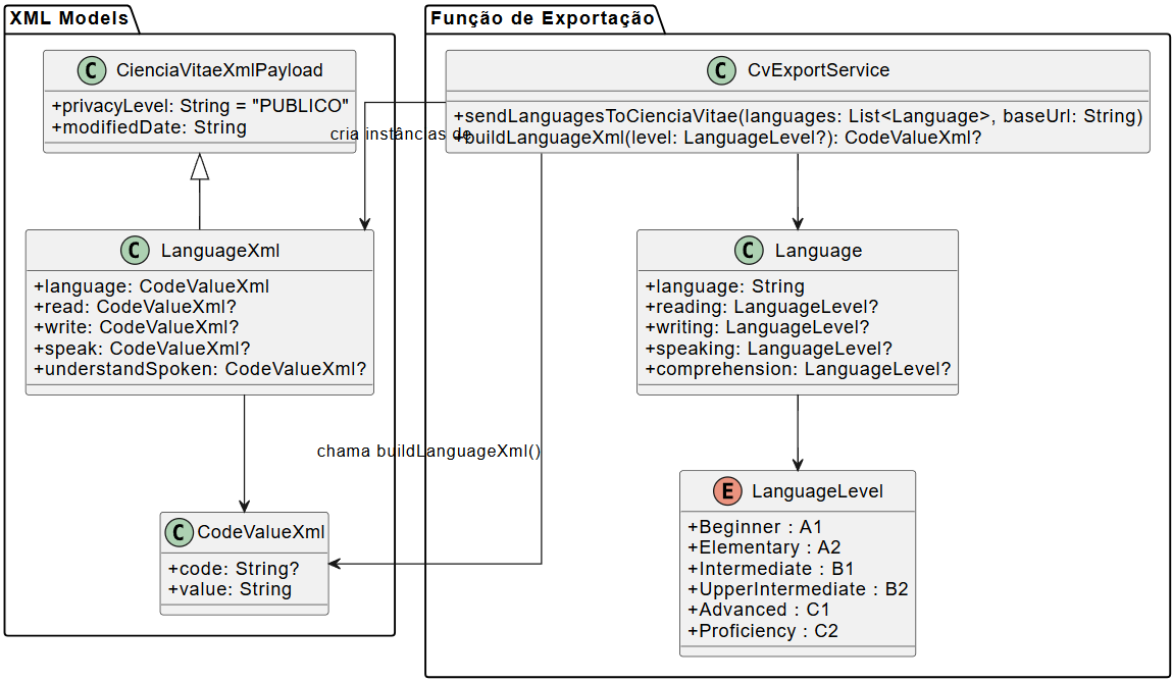


Figura 5.6: Diagrama da exportação de competências linguísticas para a CIÊNCIAVITAE.

Comunicação com a API da CIÊNCIAVITAE: A função *postToCienciaVitae* é responsável por converter o *payload* em *XML*, adicionar o cabeçalho da declaração *XML* e delegar o envio à função *postRequest*, que utiliza o *WebClient* do *Spring Framework* para realizar a requisição *HTTP POST*. Para além disso, utiliza a função *checkResponse* que valida o *status* da resposta, para detetar erros do lado do cliente (*4xx*) ou do servidor (*5xx*), lançando exceções com mensagens específicas para facilitar o diagnóstico.

Vantagens da abordagem adotada:

- **Reutilização de código** – Componentes auxiliares como *CodeValueXml* e *CienciaVitaeXmlPayload* são utilizados por múltiplos tipos de dados, promovendo consistência e prevenindo duplicação de código.

- **Compatibilidade** – Utilização de *Jackson XML* permite gerar *XMLs* totalmente compatíveis com os esquemas definidos pela *API* da *CIÊNCIAVITAE*.

- **Robustez** – O tratamento explícito de respostas *HTTP* permite detetar rapidamente erros de integração com a *API*.

Esta integração permite que os dados curriculares armazenados na aplicação sejam facilmente enviados para a plataforma *CIÊNCIAVITAE*, de forma automática via *API*, reduzindo a necessidade de duplicação manual de informação. É, portanto, uma funcionalidade essencial para garantir a atualização sincronizada de currículos científicos em múltiplas plataformas.

5.5 *Frontend Web*

A interface gráfica da aplicação foi desenvolvida com o objetivo de oferecer uma navegação simples, funcional e orientada ao utilizador, permitindo o acesso intuitivo às funcionalidades principais do sistema, como autenticação, importação, visualização, edição e exportação de currículos científicos.

O *frontend web* é responsável por estabelecer a comunicação com os serviços *backend* através de chamadas *HTTP*. Para isso, é utilizada a biblioteca *Axios*, que permite definir instâncias configuráveis para lidar com diferentes tipos de *endpoints* da aplicação. Esta abordagem facilita a manutenção e reutilização das funções de acesso à API, garantindo uma separação clara entre operações com e sem autenticação.

A interface principal é composta por uma barra de navegação dinâmica, que se adapta consoante o estado de autenticação do utilizador. Quando autenticado, o utilizador tem acesso direto às funcionalidades do CV e à opção de terminar sessão. Caso contrário, é-lhe apresentada a opção de iniciar sessão ou registar-se. Esta lógica de navegação é gerida de forma reativa, com base na existência de um *token* de autenticação guardado localmente.

A navegação entre páginas é gerida através da biblioteca *react-router*, permitindo alternar de forma eficiente entre as diferentes vistas da aplicação, como a página de autenticação, o painel principal e a visualização do currículo. Este mecanismo contribui para uma experiência fluida e sem recarregamentos completos da página, mantendo o estado da aplicação entre interações.

A Figura 5.7 representa o fluxo de navegação da interface do utilizador, ilustrando as principais rotas e interações possíveis na aplicação.

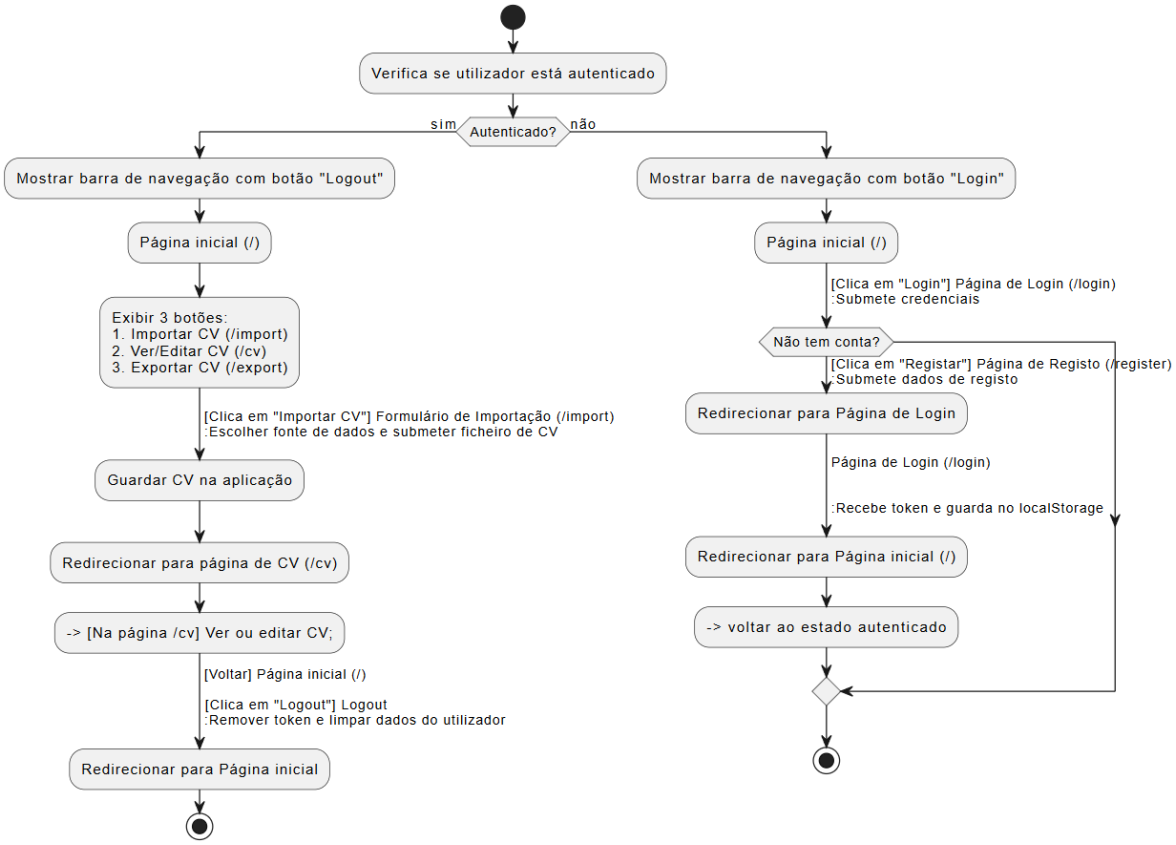


Figura 5.7: Fluxo de navegação da interface do utilizador

Este fluxo permite visualizar de forma clara a transição entre diferentes estados da aplicação. Com esta organização, assegura-se uma experiência fluida e orientada à tarefa do utilizador.

Capítulo 6

Testes

A estratégia de testes adotada na aplicação seguiu uma organização em camadas coerente com a arquitetura *Spring Boot*, incluindo testes direcionados tanto para os controladores (camada web) quanto para os serviços. Os testes foram realizados preferencialmente com interação real contra uma base de dados configurada para teste, evitando o uso de mocks para garantir maior fidelidade na verificação da lógica e persistência dos dados.

No que respeita à integração com a API da *CIÊNCIAVITAE*, foram utilizados mocks totalmente compatíveis com a interface real da API, uma vez que o acesso fornecido à API real apenas permite operações de leitura (*GET*), estando as operações de escrita (*POST*) restritas. Esta abordagem assegura que o comportamento da aplicação em contexto real será equivalente, permitindo testar a lógica de exportação mesmo sem acesso completo à API oficial.

As principais camadas testadas foram:

- **Testes de Controladores (Web Layer)**: Testes de integração utilizando o *WebTestClient*, que simulam chamadas *HTTP* reais para os *endpoints REST*. Valida-se o comportamento das APIs, os códigos *HTTP* retornados e o formato das respostas *JSON*.

- **Testes de Serviços (Service Layer)**: Testes funcionais que exercitam diretamente a lógica do serviço, utilizando uma base de dados configurada para testes. Estes testes verificam que as regras e lógica são corretamente aplicadas, incluindo validações e manipulação de entidades.

6.1 Ferramentas Utilizadas

Para além das referidas anteriormente, para os testes foram utilizadas as seguintes ferramentas:

- **JUnit 5**: *Framework* base para a implementação e execução dos testes.

- **Spring Boot Test**: Facilita a configuração do contexto da aplicação durante os testes, incluindo o carregamento da base de dados em memória.

6.2 Conclusão

Esta abordagem de testes funcionais, que privilegia a execução contra componentes reais da aplicação — incluindo base de dados de teste — permitiu assegurar um elevado grau de confiança na correção das funcionalidades implementadas. A estratégia contribuiu para detetar antecipadamente possíveis erros, facilitar a manutenção do código e garantir a estabilidade da aplicação.

Capítulo 7

Conclusão

Este projeto teve como objetivo principal o desenvolvimento de uma aplicação web para facilitar a gestão e a interoperabilidade de currículos científicos, com enfoque na plataforma *CIÊNCIAVITAE*. Ao longo do trabalho, foi possível concretizar todos os requisitos funcionais definidos como obrigatórios, incluindo a importação de dados da plataforma *Lattes*, a sua conversão para um modelo interno de currículo científico, a edição através de uma interface gráfica, a exportação através da API da *CIÊNCIAVITAE*, bem como o armazenamento estruturado da informação.

A arquitetura modular e extensível concebida permite ainda, a futura integração com outras plataformas de gestão de currículos, o que reforça a escalabilidade e utilidade prática da solução desenvolvida. A separação clara entre componentes e o uso de tecnologias modernas asseguram a manutenibilidade do sistema, enquanto o uso de testes garante a sua robustez.

Trabalho Futuro

Apesar dos avanços conseguidos, há espaço para melhorias e extensões futuras que poderão reforçar ainda mais o valor da aplicação. Algumas das direções a explorar incluem:

1. **Importação direta a partir da *CIÊNCIAVITAE*, via API:** Embora prevista como requisito opcional, a funcionalidade de permitir a importação de currículos científicos da plataforma *CIÊNCIAVITAE* para o sistema, utilizando a sua *API* oficial não foi implementada na versão atual. A sua concretização permitiria a sincronização bidirecional dos dados e uma maior flexibilidade para os utilizadores.

2. **Suporte a múltiplas plataformas externas:** a arquitetura do sistema permite já a integração com outras fontes de dados. Assim, seria possível alargar a aplicação para suportar outras plataformas de currículos científicos.

3. **Histórico de versões de currículos:** a possibilidade de manter um histórico das alterações realizadas aos currículos permitiria aos utilizadores consultar versões anteriores e recuperar informação perdida.

4. **Migração de Kotlin para Java:** embora a aplicação *backend* tenha sido desenvolvida em *Kotlin*, poderá ser vantajoso, no futuro, migrar para *Java*, dado que esta é a linguagem mais comum em contextos institucionais e poderá facilitar a integração com sistemas existentes da própria *CIÊNCIAVITAE*, que aparenta utilizar *Java* internamente.

Em suma, o trabalho desenvolvido comprova a viabilidade técnica e a relevância prática de uma aplicação de interface de dados para currículos científicos, que responda às limitações atuais de interoperabilidade entre plataformas. A aplicação desenvolvida constitui uma base sólida para futuras evoluções, que poderão ampliar ainda mais o seu impacto no ecossistema académico e científico.

Referências

[1] Fundação para a Ciência e Tecnologia (FCT). CIÊNCIAVITAE. [Online; acedido a 06-Julho-2025].

[2] CNPq. Plataforma Lattes. [Online; acedido a 06-Julho-2025].

[3] Fundação para a Ciência e Tecnologia (FCT). Documentação da API da CIÊNCIAVITAE. [Online; acedido a 06-Julho-2025].

[4] ORCID. ORCID: Open Researcher and Contributor ID. [Online; acedido a 06-Julho-2025].

[5] FCCN - Fundação para a Computação Científica Nacional. PTCRIS: Plataforma de Gestão Integrada da Informação Científica e Tecnológica. [Online; acedido a 06-Julho-2025].

[6] World Wide Web Consortium (W3C). Extensible Markup Language (XML). [Online; acedido a 06-Julho-2025].

[7] Douglas Crockford. JSON - JavaScript Object Notation. [Online; acedido a 06-Julho-2025].

[8] EuroCRIS. Common European Research Information Format (CERIF). [Online; acedido a 06-Julho-2025].

[9] Fundação para a Ciência e a Tecnologia (FCT). Fundação para a Ciência e a Tecnologia. [Online; acedido a 06-Julho-2025].

[10] Fundação para a Ciência e a Tecnologia (FCT). SciPROJ – Sistema de Gestão de Projetos Científicos. [Online; acedido a 06-Julho-2025].

[11] FCCN - Fundação para a Computação Científica Nacional. RCAAP – Repositório Científico de Acesso Aberto de Portugal. [Online; acedido a 06-Julho-2025].

[12] DGEEC – Direção-Geral de Estatísticas da Educação e Ciência. PRIES – Plataforma de Recolha de Informação do Ensino Superior. [Online; acedido a 06-Julho-2025].

[13] DGEEC – Direção-Geral de Estatísticas da Educação e Ciência. RENATES – Rede Nacional de Testbeds. [Online; acedido a 06-Julho-2025].

[14] Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq). Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq). [Online; acedido a 06-Julho-2025].

[15] JetBrains. Kotlin Documentation. [Online; acedido a 06-Julho-2025].

[16] Oracle Corporation. The Java Language. [Online; acedido a 06-Julho-2025].

[17] VMware. Spring Framework. [Online; acedido a 06-Julho-2025].

[18] VMware. Spring Boot Reference Documentation. [Online; acedido a 06-Julho-2025].

[19] Spring Security Team. BCryptPasswordEncoder (Spring Security API). [Online; acedido a 06-Julho-2025].

[20] Spring Team. Spring WebClient Documentation. [Online; acedido a 06-Julho-2025].

[21] FasterXML. Jackson XML Dataformat, 2023. [Online; acedido a 06-Julho-2025].

[22] PostgreSQL Global Development Group. PostgreSQL 17.5 Documentation, 2025. [Online; acedido a 06-Julho-2025].

[23] Spring Team. Spring Data JPA Documentation. [Online; acedido a 06-Julho-2025].

[24] Microsoft. TypeScript Documentation. [Online; acedido a 06-Julho-2025].

[25] Mozilla Developer Network (MDN). JavaScript Guide. [Online; acedido a 06-Julho-2025].

[26] Meta (Facebook). React – A JavaScript Library for Building User Interfaces. [Online; acedido a 06-Julho-2025].

[27] Remix Software. React Router Documentation. [Online; acedido a 06-Julho-2025].

[28] Docker, Inc. Docker: Empowering app development for developers. [Online; acedido a 06-Julho-2025].

[29] Docker, Inc. Docker Compose: Define and run multi-container applications. [Online; acedido a 06-Julho-2025].

[30] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014. [Online; acedido a 06-Julho-2025].

[31] GitHub, Inc. GitHub: Where the World Builds Software. [Online; acedido a 06-Julho-2025].

[32] Postman, Inc. Postman API Platform. [Online; acedido a 06-Julho-2025].

[33] GNOME Project. Dia Diagram Editor. [Online; acedido a 06-Julho-2025].

[34] diagrams.net. diagrams.net. [Online; acedido a 06-Julho-2025].

Apêndice A

Base de Dados

RI	Descrição
1	O valor do campo deve ser `Personal` ou `Professional`.
2	O valor do campo `Phone.device` deve ser um de: `Telephone`, `Mobile`, `Fax`, `Pager`.
3	O valor do campo `Website.type` deve ser um de: `Blog`, `Feed`, `Personal`, `SocialMedia`, `Professional`, `Scholar`.
4	O valor dos níveis de competência linguística deve ser um de: `Beginner`, `Elementary`, `Intermediate`, `UpperIntermediate`, `Advanced`, `Proficiency`.
5	O valor do campo `Identifier.type` deve ser um de: `ORCID`, `WOS`, `SCOPUS`, `GOOGLE`, `AUTHENTICUSID`, `LATTESID`.
6	O valor do campo `Education.degree` deve ser um de: `PrimaryEducation`, `SecondaryEducation`, `Bachelor`, `Postgraduate`, `Master`, `Doctorate`, `PostDoctorate`, `Other`.
7	O valor do campo `Education.status` deve ser um de: `Ongoing`, `Attended`, `Concluded`.
8	O valor do campo `Distinction.type` deve ser um de: `Award`, `Title`, `Other`.
9	O valor do campo `Project.fundingType` deve ser um de: `Grant`, `Other`, `Contract`.
10	O valor do campo `Project.status` deve ser um de: `Concluded`, `Cancelled`, `Ongoing`.
11	O valor do campo deve ser `Supervisor` ou `CoSupervisor`.
12	O valor do campo deve ser um de: `MASTERS`, `DOCTORATE`, `POSTDOC`, `SPECIALIZATION`, `RESEARCH`, `BACHELOR`, `OTHER`

Tabela A.1: Restrições de Integridade aplicadas no modelo de dados

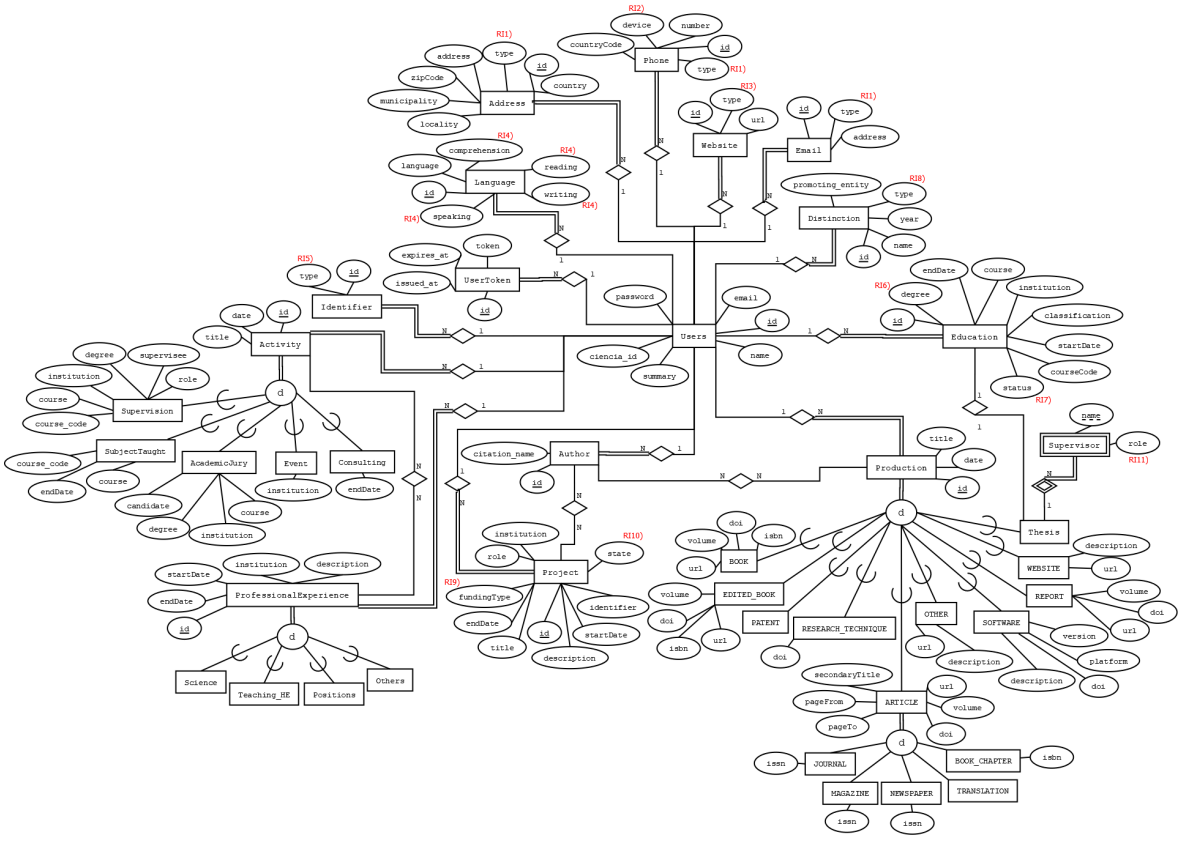


Figura A.1: Diagrama Entidade-Associação do projeto.